

Git և GitHub մանրամասն դասախոսության նյութ

Ինքնուրույն ուսուցում և առաջին լաբորատոր աշխատանք

Նյութը նախատեսված է Git և GitHub չօգտագործած ուսանողների համար: Այն բացատրում է հասկացությունները պարզ լեզվով, ցույց է տալիս Windows-ում տեղադրման ամբողջ ընթացքը և քայլ առ քայլ առաջնորդում է մինչև առաջին HTML ֆայլի commit-ն ու GitHub push-ը: Ուսանողը կարող է նյութն անցնել ինքնուրույն՝ յուրաքանչյուր քայլից հետո կատարելով նշված ստուգումը:

Ուսուցման արդյունքներ

- տարբերակել Git ծրագիրը և GitHub առցանց հարթակը
- տեղադրել Git for Windows-ը և ստուգել աշխատանքը terminal-ում
- կարգավորել commit-ի հեղինակի անունը, email-ը և main branch-ի անվանումը
- ստեղծել repository, տարբերակել working directory, staging area և commit հասկացությունները
- ստեղծել ու պահպանել առաջին HTML ֆայլը add և commit հրամաններով
- ստեղծել դատարկ GitHub repository, ավելացնել origin remote և կատարել push
- կարդալ git status, git diff և git log հրամանների արդյունքները
- գտնել և ուղղել սկսնակների տարածված սխալները

Նյութի կառուցվածք

Բաժին	Բովանդակություն
1	Տարբերակների վերահսկում, Git և GitHub
2	Repository, commit, branch և Git-ի աշխատանքային տարածքներ
3	Git-ի տեղադրում և առաջին կարգավորում Windows-ում
4	GitHub հաշիվ, repository և անվտանգ նույնականացում
5	Լաբորատոր աշխատանք 1՝ առաջին HTML ֆայլից մինչև push
6	Ամենօրյա աշխատանքային ցիկլ, .gitignore և անվտանգություն
7	Սխալների լուծում, հրամանների հուշաթերթ և գնահատում

1 Տարբերակների վերահսկման գաղափարը

Ծրագրի ֆայլերը աշխատանքի ընթացքում անընդհատ փոխվում են: Առանց տարբերակների վերահսկման ուսանողը հաճախ ստեղծում է project-final, project-final-2 կամ project-last անուններով պատճեններ և հետո դժվարությամբ է հասկանում, թե որ տարբերակն է ճիշտ: Տարբերակների վերահսկման համակարգը պահում է փոփոխությունների կազմակերպված պատմություն և թույլ է տալիս տեսնել, թե երբ, ով և ինչ է փոխել:

Git-ը բաշխված տարբերակների վերահսկման համակարգ է: Յուրաքանչյուր աշխատանքային համակարգիչ կարող է ունենալ repository-ի ֆայլերը և ամբողջ պատմությունը: Տեղային commit կատարելու համար ինտերնետ անհրաժեշտ չէ. ինտերնետը պահանջվում է remote repository-ի հետ տվյալներ փոխանակելիս:

1.1 Git և GitHub տարբերությունը

Հատկություն	Git	GitHub
Ի՞նչ է	համակարգչում աշխատող ծրագիր	Git repository-ների առցանց հաղթակ
Հիմնական դեր	պահում է փոփոխությունների պատմությունը	պահում և տարածում է repository-ն համացանցում
Ինտերնետ	տեղային աշխատանքի համար պարտադիր չէ	push, pull և կայքից օգտվելու համար պետք է
Օրինակ	git add, git commit, git log	repository էջ, Issues, Pull Requests

GitHub-ը չի փոխարինում Git-ին: Git-ը կառավարում է տեղային պատմությունը, իսկ GitHub-ը պահում է դրա remote պատճենը և տալիս համագործակցության լրացուցիչ գործիքներ: Առաջին լաբորատոր աշխատանքում սկզբում կստեղծվի տեղային Git repository, հետո այն կկապվի GitHub repository-ի հետ:

1.2 Ինչ է repository-ն

Repository-ն նախագծի ֆայլերի և դրանց Git պատմության ամբողջությունն է: Տեղային repository-ն գտնվում է համակարգչում, իսկ remote repository-ն՝ GitHub-ում կամ այլ սերվերում: git init հրամանը ընթացիկ պանակում ստեղծում է .git թաքնված պանակ, որտեղ Git-ը պահում է պատմության և կարգավորումների տվյալները: .git պանակը ձեռքով փոխել կամ ջնջել պետք չէ:

2 Git-ի հիմնական հասկացությունները

Տերմին	Պարզ բացատրություն
working directory	Նախագծի իրական ֆայլերը, որոնք խմբագրում ենք
untracked	Նոր ֆայլ, որի մասին Git-ը դեռ պատմություն չունի
modified	հետևվող ֆայլ, որը փոխվել է վերջին commit-ից հետո
staging area	հաջորդ commit-ի համար ընտրված փոփոխությունների միջանկյալ տարածք
commit	Նախագծի պահված վիճակ՝ հեղինակով, ժամանակով և հաղորդագրությամբ
branch	commit-ների պատմության աշխատանքային ուղղություն
HEAD	ընթացիկ branch-ի վերջին ընտրված commit-ի ցուցիչ

Տերմին	Պարզ բացատրություն
remote	այլ repository-ի հասցե, սովորաբար GitHub-ում
origin	առաջին remote-ի ընդունված կարճ անունը

2.1 Ֆայլի վիճակների փոփոխությունը

Նոր index.html ֆայլը սկզբում untracked է: git add index.html հրամանից հետո այն staged է և պատրաստ է հաջորդ commit-ի համար: git commit հրամանը staged տարբերակը պահում է տեղային պատմության մեջ: Եթե commit-ից հետո ֆայլը նորից փոխվի, այն կդառնա modified և կրկին պետք է անցնի add ու commit քայլերով:

```

untracked / modified
  git add
    ↓
  staged
  git commit
    ↓
  committed
  git push
    ↓
  GitHub repository

```

2.2 Ինչ է commit-ը

Commit-ը պարզապես Save գործողությունն է: Save-ը փոփոխում է ֆայլը համակարգչի վրա, իսկ commit-ը նախագծի ընտրված վիճակը պահում է Git պատմության մեջ: Յուրաքանչյուր commit ունի hash նույնականացուցիչ, հեղինակ, email, ժամանակ և հաղորդագրություն: Լավ commit message-ը կարճ և հստակ նկարագրում է, թե ինչ է փոխվել, օրինակ՝ Add first HTML page կամ Fix navigation links:

3 Git-ի տեղադրում Windows-ում

Git-ը պետք է ներբեռնել պաշտոնական git-scm.com կայքից: Տարբերակի համարը ժամանակի ընթացքում փոխվում է, ուստի ուսանողը ընտրում է պաշտոնական էջում առաջարկվող վերջին պահպանվող տարբերակը: Սովորական Intel կամ AMD պրոցեսորով Windows համակարգչի համար ընտրվում է x64 installer-ը, իսկ ARM սարքի համար՝ ARM64 տարբերակը:

3.1 Տեղադրման քայլերը

1. Բացել <https://git-scm.com/install/windows> հասցեն:
2. Ներբեռնել համապատասխան Windows installer-ը:
3. Աշխատեցնել ներբեռնված ֆայլը և թույլատրել տեղադրումը, եթե Windows-ը հարցնում է:
4. Սկսնակների համար պահպանել installer-ի առաջարկվող ընտրությունները: Եթե առաջարկվում է editor ընտրել և Visual Studio Code-ը հասանելի է, կարելի է ընտրել այն:
5. Ավարտել տեղադրումը և փակել հին terminal պատուհանները:
6. Start ցանկից բացել Git Bash կամ VS Code-ում բացել նոր Terminal:

Այլընտրանքային եղանակը Windows Package Manager-ն է: PowerShell կամ Command Prompt պատուհանում կարելի է աշխատեցնել հետևյալ հրամանը:

```
winget install --id Git.Git -e --source winget
```

3.2 Տեղադրման ստուգումը

Բացել Git Bash և գրել հրամանը:

```
git --version
```

Ստուգում: Պետք է երևա git version 2.x.x տեսակի պատասխան: ճշգրիտ տարբերակի համարը կարող է այլ լինել: Եթե terminal-ը գրում է command not found կամ git is not recognized, նախ փակել ու նորից բացել terminal-ը, ապա անհրաժեշտության դեպքում վերատեղադրել Git-ը:

3.3 Terminal-ի նվազագույն հրամանները

Հրաման	Նպատակ	Օրինակ
pwd	ցույց է տալիս ընթացիկ պանակի ուղին	pwd
ls	ցույց է տալիս ընթացիկ պանակի ֆայլերը	ls
cd պանակ	անցնում է նշված պանակ	cd git-first-lab
cd ..	անցնում է մեկ մակարդակ վերև	cd ..
clear	մաքրում է terminal-ի տեսանելի մասը	clear

Git հրամանները միշտ աշխատում են terminal-ի ընթացիկ պանակի նկատմամբ: Այդ պատճառով git init կատարելուց առաջ անհրաժեշտ է pwd և ls հրամաններով ստուգել, որ terminal-ը բացված է հենց նախագծի պանակում:

4 Git-ի առաջին կարգավորումը

Git-ը յուրաքանչյուր commit-ի մեջ պահում է հեղինակի անունն ու email-ը: Առաջին commit-ից առաջ անհրաժեշտ է դրանք սահմանել մեկ անգամ տվյալ համակարգչի համար: Չակերտները պարտադիր են, երբ անունը բացատ ունի:

```
git config --global user.name "Անուն Ազգանուն"  
git config --global user.email "student@example.com"  
git config --global init.defaultBranch main
```

user.name դաշտում գրվում է մարդու ընթերցելի անունը, ոչ թե պարտադիր GitHub username-ը: user.email-ը նպատակահարմար է գրել այն հասցեով, որը կապված է GitHub հաշվի հետ: init.defaultBranch կարգավորումը նոր repository-ների առաջին branch-ի անունը դարձնում է main:

4.1 Կարգավորումների ստուգումը

```
git config --global --list
```

Ստուգում: Ցանկում գտնել user.name, user.email և init.defaultbranch տողերը: Եթե արժեքը սխալ է, նույն config հրամանը նոր ճիշտ արժեքով աշխատեցնելը կփոխարինի հին արժեքը:

4.2 Global և local կարգավորում

--global տարբերակը կարգավորումը կիրառում է տվյալ օգտատիրոջ բոլոր repository-ների համար: Առանձին նախագծի համար այլ անուն կամ email սահմանելու դեպքում հրամանը կատարվում է repository-ի ներսում՝ առանց --global տարբերակի: local կարգավորումը տվյալ repository-ում առավելություն ունի global արժեքի նկատմամբ:

```
git config user.email "other@example.com"
git config --local --list
```

5 GitHub հաշիվ և անվտանգություն

GitHub-ում անհրաժեշտ է ստեղծել հաշիվ և հաստատել email-ը: Username-ը դառնում է repository-ների URL-ի մաս, օրինակ՝ <https://github.com/username/project>: Խորհուրդ է տրվում միացնել երկբայլ նույնականացումը և recovery codes-ը պահել անվտանգ վայրում:

Command line-ից GitHub հասանելի է HTTPS կամ SSH կապով: Առաջին լաբորատոր աշխատանքի համար օգտագործվում է HTTPS, քանի որ Git for Windows-ի հետ Git Credential Manager-ը սովորաբար կարող է բացել բրաուզերի մուտքը և անվտանգ պահել նույնականացումը: GitHub-ի սովորական account password-ը Git գործողությունների HTTPS password դաշտում չի կիրառվում: Եթե բրաուզերի կամ Credential Manager-ի մուտքը հասանելի չէ, օգտագործվում է personal access token կամ GitHub CLI:

5.1 Repository visibility

Տեսակ	Ով կարող է տեսնել	Երբ ընտրել
Public	համացանցի բոլոր օգտատերերը	բաց ուսումնական աշխատանք և portfolio
Private	միայն owner-ը և հրավիրված մադրիկ	դասախոսի պահանջ, չհրապարակվող աշխատանք

Public repository-ում երբեք չի կարելի տեղադրել գաղտնաբառ, access token, API key, անձնական փաստաթուղթ կամ այլ գաղտնի տվյալ: Private repository-ն նույնպես գաղտնիքների անվտանգ պահեստ չէ: Գաղտնի արժեքները պահվում են նախագծից դուրս կամ environment variables-ի միջոցով:

6 Լաբորատոր աշխատանք 1

Թեմա՝ Git-ի տեղադրում, առաջին repository-ի ստեղծում, HTML ֆայլի commit և GitHub push: Աշխատանքը նախատեսված է մեկ ուսանողի ինքնուրույն կատարման համար: Յուրաքանչյուր քայլից հետո պետք է ստուգել արդյունքը և միայն դրանից հետո անցնել հաջորդ քայլին:

6.1 Նպատակ և վերջնական արդյունք

- Windows-ում տեղադրված և աշխատող Git:
- Կարգավորված user.name, user.email և main branch:
- git-first-lab անունով տեղային repository:
- ճիշտ HTML կառուցվածքով index.html ֆայլ:
- Add first HTML page հաղորդագրությամբ առաջին commit:
- GitHub-ում git-first-lab repository, որտեղ երևում են ֆայլն ու commit-ը:

6.2 Քայլ 1 Նախագծի պանակի ստեղծումը

1. Documents պանակում ստեղծել git-first-lab անունով նոր պանակ:
2. Բացել Visual Studio Code-ը և ընտրել File, Open Folder:
3. Ընտրել git-first-lab պանակը և հաստատել բացումը:
4. Explorer բաժնում ստուգել, որ բացված է հենց git-first-lab պանակը:

Ստուգում: VS Code-ի Explorer-ում վերևում պետք է երևա GIT-FIRST-LAB անունը, իսկ պանակը դեռ կարող է դատարկ լինել:

6.3 Քայլ 2 Առաջին HTML ֆայլը

git-first-lab պանակում ստեղծել index.html ֆայլ և գրել հետևյալ օրինակը: Ուսանողը կարող է փոխել էջի տեսանելի տեքստը, բայց պետք է պահպանի ճիշտ HTML կառուցվածքը:

```
<!doctype html>
<html lang="hy">
<head>
  <meta charset="utf-8">
  <title>Առաջին Git աշխատանք</title>
</head>
<body>
  <h1>Իմ առաջին commit-ը</h1>
  <p>Ֆայլը ստեղծել եմ Git լաբորատոր աշխատանքի համար:</p>
</body>
</html>
```

Պահպանել ֆայլը Ctrl+S ստեղծելով: File Explorer-ից կրկնակի սեղմելով index.html ֆայլը՝ բացել այն բրաուզերում և ստուգել բովանդակությունը:

Ստուգում: Բրաուզերում պետք է երևան «Իմ առաջին commit-ը» վերնագիրը և մեկ պարբերություն: Եթե երևում է կողք, հնարավոր է ֆայլը պահպանվել է index.html.txt անունով. Windows-ում միացնել file name extensions ցուցադրումը և ուղղել անունը:

6.4 Քայլ 3 Terminal-ը ճիշտ պանակում

VS Code-ում ընտրել Terminal, New Terminal: Եթե բացված shell-ը PowerShell է, Git հրամանները նույնպես պետք է աշխատեն: Կարելի է terminal-ի ընտրիչից բացել Git Bash:

```
pwd
ls
```

Ստուգում: pwd հրամանի վերջում պետք է երևա git-first-lab, իսկ ls հրամանի արդյունքում՝ index.html: Եթե ոչ, cd հրամանով անցնել ճիշտ պանակ կամ VS Code-ում նորից բացել ճիշտ folder-ը:

6.5 Քայլ 4 Repository-ի սկզբնավորումը

```
git init
git status
```

git init-ը ընթացիկ պանակում ստեղծում է նոր Git repository: Այն ստեղծում է .git թաքնված պանակ, բայց չի ստեղծում commit և ոչինչ չի ուղարկում GitHub: git status-ը պետք է ցույց տա, որ ընթացիկ branch-ը main է և index.html ֆայլը untracked է:

Ստուգում: Terminal-ում գտնել Untracked files բաժինը և դրա տակ index.html անունը: Եթե branch-ը master է, աշխատեցնել git branch -M main հրամանը կամ ստուգել init.defaultBranch կարգավորումը:

6.6 Քայլ 5 Ֆայլի ընտրումը staging area-ի համար

```
git add index.html
git status
```

git add index.html հրամանը index.html-ի ընթացիկ տարբերակը տեղափոխում է staging area: Այն դեռ commit չէ և դեռ GitHub-ում չի երևում: git status-ի երկրորդ կանչը պետք է ցույց տա Changes to be committed և new file: index.html:

Ստուգում: index.html անունը սովորաբար երևում է կանաչ գույնով Changes to be committed բաժնում: Եթե այն շարունակում է մնալ Untracked files բաժնում, ստուգել հրամանի և ֆայլի անվան գրությունը:

6.7 Քայլ 6 Առաջին commit-ը

```
git commit -m "Add first HTML page"
```

Commit message-ը պետք է նկարագրի փոփոխությունը, ոչ թե լինի first, work կամ test տեսակի անորոշ բառ: Այս օրինակում Add first HTML page արտահայտությունը հստակ ցույց է տալիս, որ commit-ը ավելացրել է առաջին HTML էջը: Commit-ից հետո terminal-ը ցույց է տալիս commit-ի կարճ hash-ը և փոփոխված ֆայլերի քանակը:

```
git status
git log --oneline
```

Ստուգում: git status-ը պետք է գրի nothing to commit, working tree clean: git log --oneline-ում պետք է երևա մեկ տող՝ տարբեր hash-ով և Add first HTML page հաղորդագրությամբ:

6.8 Քայլ 7 Դատարկ GitHub repository-ի ստեղծումը

1. Բրաուզերով մուտք գործել github.com:
2. Վերևի աջ մասում սեղմել + և ընտրել New repository:
3. Owner դաշտում ընտրել սեփական հաշիվը:

4. Repository name դաշտում գրել git-first-lab:
5. Description դաշտում գրել My first Git and GitHub laboratory work:
6. Ընտրել Public կամ դասախոսի պահանջած Private տարբերակը:
7. Չընտրել Add a README file, Add .gitignore և Choose a license տարբերակները:
8. Սեղմել Create repository:

Repository-ն ստեղծվում է դատարկ, որովհետև տեղային repository-ում արդեն կա առաջին commit-ը: Եթե GitHub-ում նախապես README կամ այլ ֆայլ ստեղծվի, remote repository-ն կունենա առանձին commit, և սկսնակի առաջին push-ը կալող է բախվել պատմությունների տարբերության հետ:

Ստուգում: Պետք է բացվի Quick Setup էջը, որտեղ GitHub-ը ցուցադրում է HTTPS հասցեն: Օրինակ՝ <https://github.com/USERNAME/git-first-lab.git>:

6.9 Քայլ 8 GitHub repository-ի կապումը origin անունով

GitHub-ի Quick Setup էջից պատճենել HTTPS URL-ը: Terminal-ում համոզվել, որ դեռ գտնվում եք git-first-lab պանակում, և կատարել հիմնական ռեմոտ-URL-ի փոխարեն տեղադրելով պատճենված հասցեն:

```
git remote add origin REMOTE-URL
git remote -v
```

origin-ը remote repository-ի պայմանական կարճ անունն է: git remote -v-ը պետք է ցույց տա նույն URL-ը fetch և push գործողությունների համար:

Ստուգում: URL-ում պետք է լինի հենց ուսանողի username-ը և git-first-lab անունը: Սխալ URL-ի դեպքում աշխատեցնել git remote set-url origin CORRECT-URL:

6.10 Քայլ 9 Առաջին push-ը

```
git push -u origin main
```

push հրամանը տեղային main branch-ի commit-ները ուղարկում է origin remote repository: -u տարբերակը ստեղծում է կապ տեղային main և remote origin/main branch-երի միջև: Հաջորդ փոփոխություններից հետո նույն branch-ից բավարար կլինի գրել git push:

Առաջին push-ի ժամանակ Git Credential Manager-ը կալող է բացել GitHub-ի մուտքի պատուհան կամ բրաուզեր: Ընտրել Sign in with your browser, մուտք գործել այն GitHub հաշվով, որտեղ ստեղծվել է repository-ն, և հաստատել թույլտվությունը: Եթե terminal-ը խնդրում է password և բրաուզերի մուտք չկա, GitHub account password-ը չի օգտագործվում. պետք է կարգավորել credential helper, GitHub CLI կամ personal access token:

Ստուգում: Հաջող push-ից հետո terminal-ում սովորաբար երևում են main -> main և branch main set up to track origin/main գրառումները: Authentication failed սխալի դեպքում ստուգել GitHub հաշիվը և Windows Credential Manager-ում պահպանված հին տվյալները:

6.11 Քայլ 10 Արդյունքի ստուգումը GitHub-ում

1. Բրաուզերում թարմացնել git-first-lab repository-ի էջը:
2. Ստուգել, որ ֆայլերի ցանկում երևում է index.html:
3. Սեղմել index.html և ստուգել HTML կոդը:
4. Բացել Commits բաժինը և գտնել Add first HTML page հաղորդագրությունը:

5. Ստուգել, որ branch ընտրիչում main է:

6. Պատճենել repository-ի գլխավոր URL-ը հանձնելու համար:

GitHub-ի ֆայլերի դիտումը ցույց է տալիս source code-ը, ոչ թե պատրաստ վեբ էջի տեսքը: GitHub Pages-ը առանձին ծառայություն է, որը կարելի է ուսումնասիրել հետագա դասերում: Այս աշխատանքի նպատակը Git պատմության և push գործընթացի յուրացումն է:

7 Երկրորդ commit-ի փորձը

Առաջին push-ից հետո կատարել ևս մեկ փոքր փոփոխություն, որպեսզի ամրապնդվի ամբողջ աշխատանքային ցիկլը: index.html ֆայլում ավելացնել նոր պարբերություն, պահպանել այն և terminal-ում հերթով կատարել հրամանները:

```
git status
git diff
git add index.html
git status
git commit -m "Update page content"
git log --oneline
git push
```

git diff-ը ցույց է տալիս working directory-ի և staging area-ի տարբերությունը: Կանաչ + նշանով տողերը ավելացված են, կարմիր - նշանով տողերը՝ հեռացված: git add-ից հետո սովորական git diff-ը կարող է դատարկ լինել, որովհետև փոփոխությունն արդեն staged է: Staged տարբերությունը դիտելու համար կարելի է օգտագործել git diff --staged հրամանը:

Ստուգում: GitHub-ի Commits բաժնում պետք է երևան արդեն երկու commit: Ամենավերևում պետք է լինի Update page content, իսկ ներքևում՝ Add first HTML page:

8 Ամենօրյա աշխատանքային ցիկլը

Քայլ	Հրաման	Հարց
1	git status	ո՞ր branch-ում եմ և ի՞նչ է փոխվել
2	git diff	ո՞ր տողերն են փոխվել
3	git add ֆայլ	ո՞ր փոփոխությունն է մտնելու commit
4	git diff --staged	ի՞նչ է արդեն staged
5	git commit -m "..."	ինչպե՞ս պահել ընտրված վիճակը
6	git push	ինչպե՞ս ուղարկել commit-ը GitHub

Կարևոր սովորություն է commit անել փոքր, իմաստավոր ամբողջությունների համար: Եթե մեկ commit-ում միաժամանակ կան կապ չունեցող տասնյակ փոփոխություններ, հետո դժվար է հասկանալ պատմությունը կամ վերադարձնել միայն անհրաժեշտ մասը: Commit message-ը պետք է համապատասխանի իրական փոփոխությանը:

8.1 git add index.html և git add . տարբերությունը

git add index.html հրամանը staging area է տեղափոխում միայն նշված ֆայլը: git add . հրամանը ընտրում է ընթացիկ պանակի և ներքին պանակների բոլոր փոփոխությունները:

Սկսնակին խորհուրդ է տրվում նախ կատարել `git status` և հնարավորության դեպքում ավելացնել `$այլերը` անունով, որպեսզի պատահաբար չներառվեն ավելորդ կամ գաղտնի `$այլեր`:

8.2 init և clone տարբերությունը

Չրաման	Երբ կիրառել	Ինչ է ստացվում
<code>git init</code>	Նախագծի պանակն արդեն համակարգչում է	Նոր տեղային repository
<code>git clone URL</code>	repository-ն արդեն կա GitHub-ում	<code>\$այլերը</code> , ամբողջ պատմությունը և origin կապը

Առաջին լաբորատոր աշխատանքում օգտագործվում է `git init`, որովհետև նախագիծը նախ ստեղծվում է համակարգչում: Նույն նախագիծն ուրիշ համակարգչում շարունակելու համար արդեն պետք է օգտագործել `git clone repository-URL`: Նույն պանակում `init` և `clone` հրամանները միասին կիրառելու կարիք չկա:

8.3 fetch, pull և push

Չրաման	Ուղղություն	Գործողություն
<code>git fetch</code>	GitHub-ից local	բերում է remote տեղեկությունը՝ առանց միացնելու ընթացիկ <code>\$այլերին</code>
<code>git pull</code>	GitHub-ից working directory	բերում և միացնում է remote փոփոխությունները
<code>git push</code>	local-ից GitHub	ուղարկում է տեղային commit-ները

9 README և .gitignore

`README.md` `$այլը` ներկայացնում է նախագիծը՝ նպատակը, օգտագործման եղանակը, գործարկման քայլերը և հեղինակին վերաբերող անհրաժեշտ տեղեկությունը: Markdown ձևաչափը թույլ է տալիս գրել վերնագրեր, ցուցակներ, հղումներ և code blocks: `README`-ն կարելի է ստեղծել տեղային repository-ում և ավելացնել որպես հաջորդ `commit`:

```
# Git First Lab

Իմ առաջին Git և GitHub լաբորատոր աշխատանքը:

## $այլեր
- `index.html` — գլխավոր էջ
```

`.gitignore` `$այլը` նշում է այն `$այլերն` ու պանակները, որոնց `Git`-ը չպետք է հետևի: Այն գործում է `untracked` `$այլերի` համար: Եթե գաղտնի `$այլն` արդեն `commit` է արվել, միայն `.gitignore` ավելացնելը այն պատմությունից չի հեռացնում:

```
# .gitignore օրինակ
.env
node_modules/
.vscode/
*.log
```

10 Անվտանգ աշխատանքի կանոններ

- Repository-ում չպահել password, token, API key կամ բանկային ու անձնական տվյալներ:
- Push-ից առաջ կատարել git status և ստուգել staged ֆայլերի ցանկը:
- Public repository-ի բոլոր ֆայլերը և commit-ների պատմությունը համարել հրապարակային:
- Եթե գաղտնիք պատահաբար commit է արվել, անմիջապես անվավեր դարձնել կամ փոխել այդ գաղտնիքը:
- Չտեղադրել ուրիշի անձնական տվյալները կամ հեղինակային իրավունքով չթույլատրված մեծ նյութեր:
- GitHub-ի recovery codes-ը և personal access token-ը պահել repository-ից դուրս:

11 Հաճախ հանդիպող սխալներ և լուծումներ

Սխալ կամ հաղորդագրություն	Հավանական պատճառ	Լուծում
git is not recognized	Git-ը տեղադրված չէ կամ terminal-ը հին է	տեղադրել Git-ը, նորից բացել terminal-ը
not a git repository	terminal-ը repository-ի ներսում չէ	pwd և ls, ապա cd ճիշտ պանակ
Author identity unknown	user.name կամ user.email չկա	կատարել git config --global հրամանները
nothing to commit	staged փոփոխություն չկա	ստուգել git status, պահպանել ֆայլը, կատարել git add
pathspec did not match	ֆայլի անունը սխալ է	կատարել ls և օգտագործել ճիշտ անունը
remote origin already exists	origin արդեն ավելացված է	git remote -v կամ git remote set-url origin URL
src refspec main does not match any	commit չկա կամ branch-ը main չէ	կատարել commit, ստուգել git branch, անհրաժեշտության դեպքում git branch -M main
Authentication failed	սխալ հաշիվ կամ հին credential	Նորից մուտք գործել, ստուգել Credential Manager-ը
non-fast-forward	GitHub-ում կան չբերված commit-ներ	git pull, լուծել տարբերությունները, հետո push

11.1 Սխալ պանակում git init

Եթե git init-ը կատարվել է parent պանակում, Git-ը կարող է սկսել հետևել նախատեսվածից շատ ֆայլերի: Terminal-ում git rev-parse --show-toplevel հրամանը ցույց է տալիս repository-ի արմատային պանակը: Եթե դա սխալ է և դեռ կարևոր commit-ներ չկան, անհրաժեշտ է փակել terminal-ը, տեսանելի դարձնել թաքնված ֆայլերը և զգուշությամբ հեռացնել միայն սխալ տեղում ստեղծված .git պանակը, ապա ճիշտ պանակում նորից կատարել git init: Ձևչելուց առաջ պարտադիր համոզվել, որ ընտրված է հենց սխալ repository-ի .git պանակը:

```
git rev-parse --show-toplevel
```

12 Հրամանների հոլշաթերթ

Հրաման	Նպատակ
git --version	ստուգել Git-ի տեղադրումը
git config --global --list	դիտել global կարգավորումները
git init	ընթացիկ պանակում ստեղծել repository
git status	դիտել branch-ը և ֆայլերի վիճակը
git add index.html	index.html-ը տեղափոխել staging area
git add .	ավելացնել ընթացիկ պանակի բոլոր փոփոխությունները
git diff	դիտել unstaged փոփոխությունները
git diff --staged	դիտել staged փոփոխությունները
git commit -m "Message"	պահել staged փոփոխությունները commit-ում
git log --oneline	դիտել commit-ների կարճ պատմությունը
git remote add origin URL	ավելացնել GitHub repository-ի կապը
git remote -v	դիտել remote հասցեները
git push -u origin main	առաջին անգամ ուղարկել main branch-ը
git push	ուղարկել նոր commit-ները
git pull	բերել և միացնել remote փոփոխությունները
git clone URL	ներբեռնել գոյություն ունեցող repository-ն

13 Լաբորատոր աշխատանքի գնահատում

Չափանիշ	Միավոր
Git-ի տեղադրում և user.name, user.email, main կարգավորում	10
git-first-lab ճիշտ պանակ և վավեր index.html	15
git init և git status ճիշտ պանակում	15
index.html staged և առաջին commit-ը ստեղծված է	20
Commit message-ը հստակ է	10
origin հասցեն ճիշտ է և առաջին push-ը հաջող է	20
GitHub repository-ի URL-ը բացվում է, ֆայլն ու commit-ը երևում են	10

13.1 Հանձնվող նյութ

- GitHub repository-ի ամբողջական URL-ը:
- git log --oneline հրամանի արդյունքի screenshot կամ պատճենված տեքստը:
- Անհրաժեշտության դեպքում կարճ նշում այն սխալի մասին, որը հանդիպել է աշխատանքի ընթացքում, և կիրառված լուծումը:

13.2 Ինքնաստուգման հարցեր

1. Git-ը և GitHub-ը ինչո՞վ են տարբերվում:
2. git add-ը և git commit-ը ինչո՞վ են առանձին քայլեր:
3. Ինչպե՞ս կարելի է հասկանալ, որ \$այլը untracked է:
4. Commit-ից հետո ինչպե՞ս դիտել պատմությունը:
5. Ի՞նչ է origin անունը:
6. Ինչո՞վ GitHub repository-ն առաջին push-ից առաջ ստեղծեցինք առանց README-ի:
7. Ի՞նչ է անում git push -u origin main հրամանը:
8. Ինչո՞վ չի կարելի repository-ում պահել access token կամ password:
9. git init-ը և git clone-ը ե՞րբ են կիրառվում:
10. Ինչպե՞ս ստուգել, թե որ remote URL-ն է կապված repository-ի հետ:

Օգտագործված պաշտոնական աղբյուրներ

Նյութը կազմվել է Git և GitHub պաշտոնական փաստաթղթերի հիման վրա: Ինտերֆեյսի անվանումները և ծրագրերի տարբերակները կարող են ժամանակի ընթացքում փոքր փոփոխությունների ենթարկվել, սակայն ներկայացված Git հրամանների հիմնական աշխատանքային տրամաբանությունը մնում է նույնը:

- Git պաշտոնական կայք և Pro Git գիրք — <https://git-scm.com/book/en/v2>
- Git for Windows տեղադրում — <https://git-scm.com/install/windows>
- Git առաջին կարգավորում — <https://git-scm.com/book/en/v2/Getting-Started-First-Time-Git-Setup>
- Git փոփոխությունների պահպանում — <https://git-scm.com/book/en/v2/Git-Basics-Recording-Changes-to-the-Repository>
- GitHub Git կարգավորում — <https://docs.github.com/en/get-started/git-basics/set-up-git>
- GitHub նոր repository ստեղծում — <https://docs.github.com/en/repositories/creating-and-managing-repositories/creating-a-new-repository>
- Տեղային նախագիծը GitHub ավելացնելը — <https://docs.github.com/en/migrations/importing-source-code/using-the-command-line-to-import-source-code/adding-locally-hosted-code-to-github>
- GitHub նույնականացում — <https://docs.github.com/en/authentication/keeping-your-account-and-data-secure/about-authentication-to-github>
- Commit-ների push — <https://docs.github.com/en/get-started/using-git/pushing-commits-to-a-remote-repository>